Signals (Part 2)

ME 350: Design & Manufacturing II

Jeffrey Koller

Some slides adopted from Mike Umbriac, David Remy, Chinedum Okwudire, & Shorya Awtar

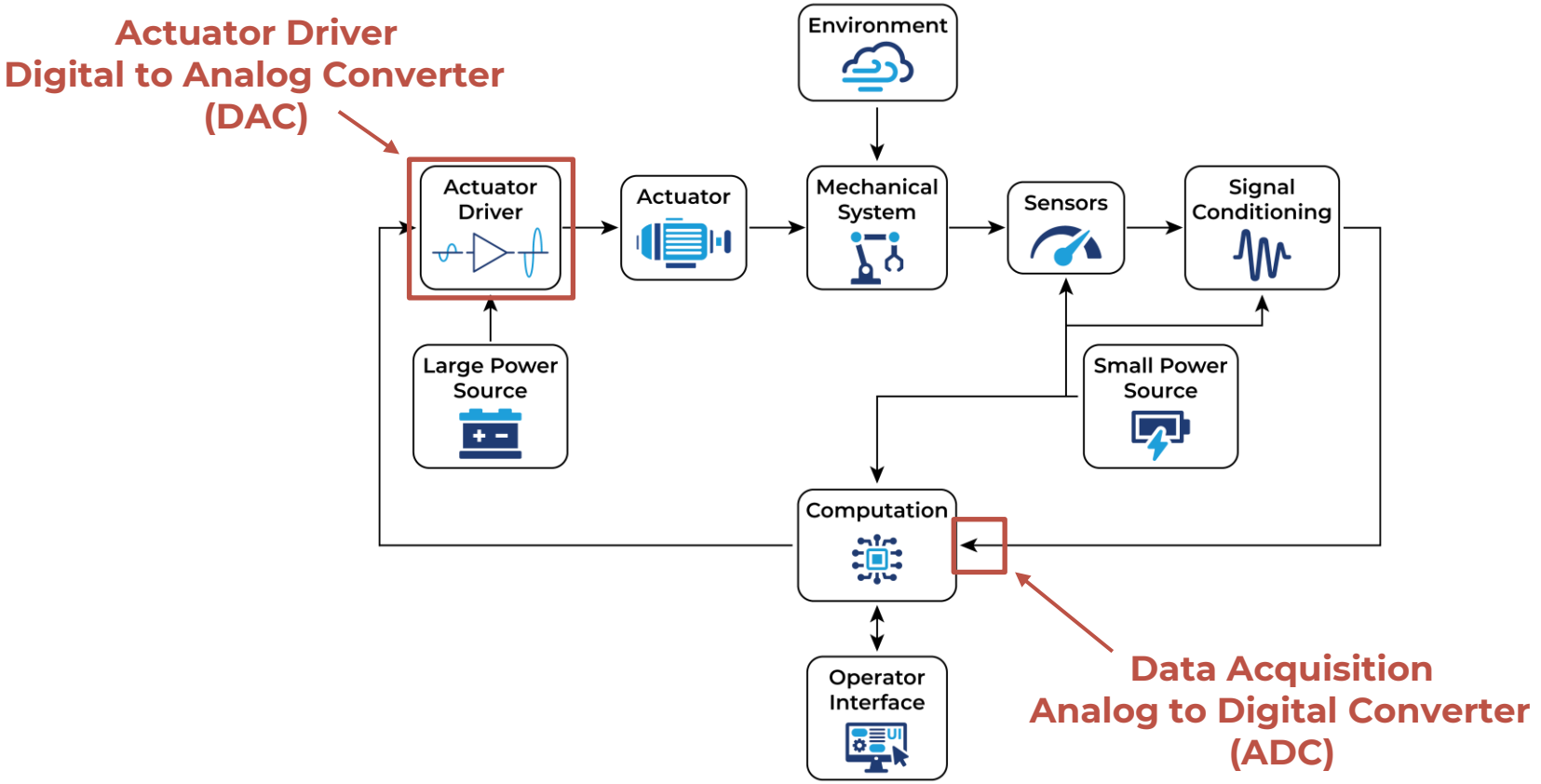
MECHANICAL ENGINEERING
UNIVERSITY OF MICHIGAN

Today's Learning Objectives

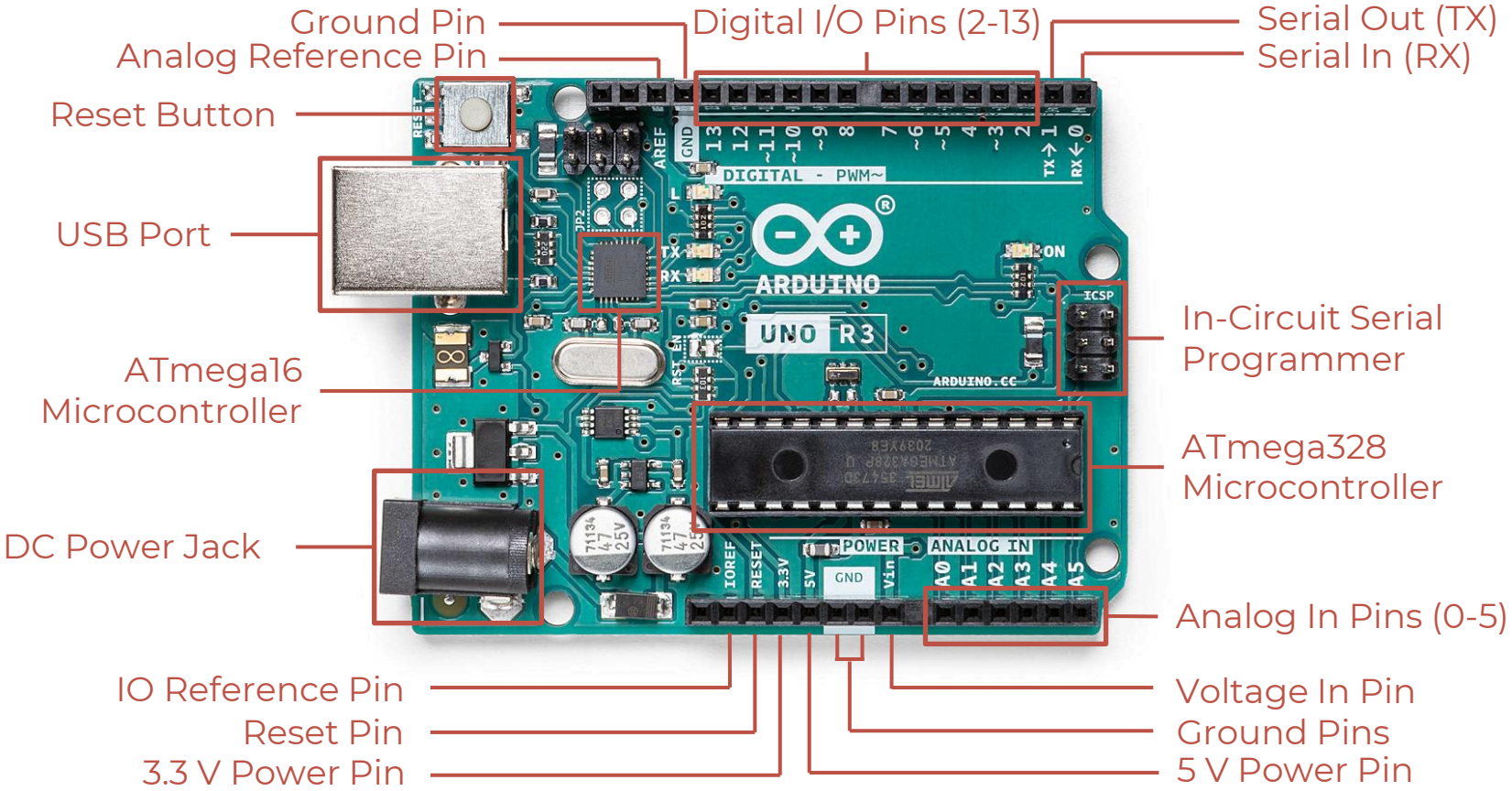
By the end of today's class you should…

- Have a high-level understanding of Analog to Digital Converters
  - Describe how ADC is achieved with Flash converters
- Have a high-level understanding of Digital to Analog Converters
  - Describe how DAC is achieved via pulse width modulation (PWM)
- Apply understanding of PWM to your ME350 Project

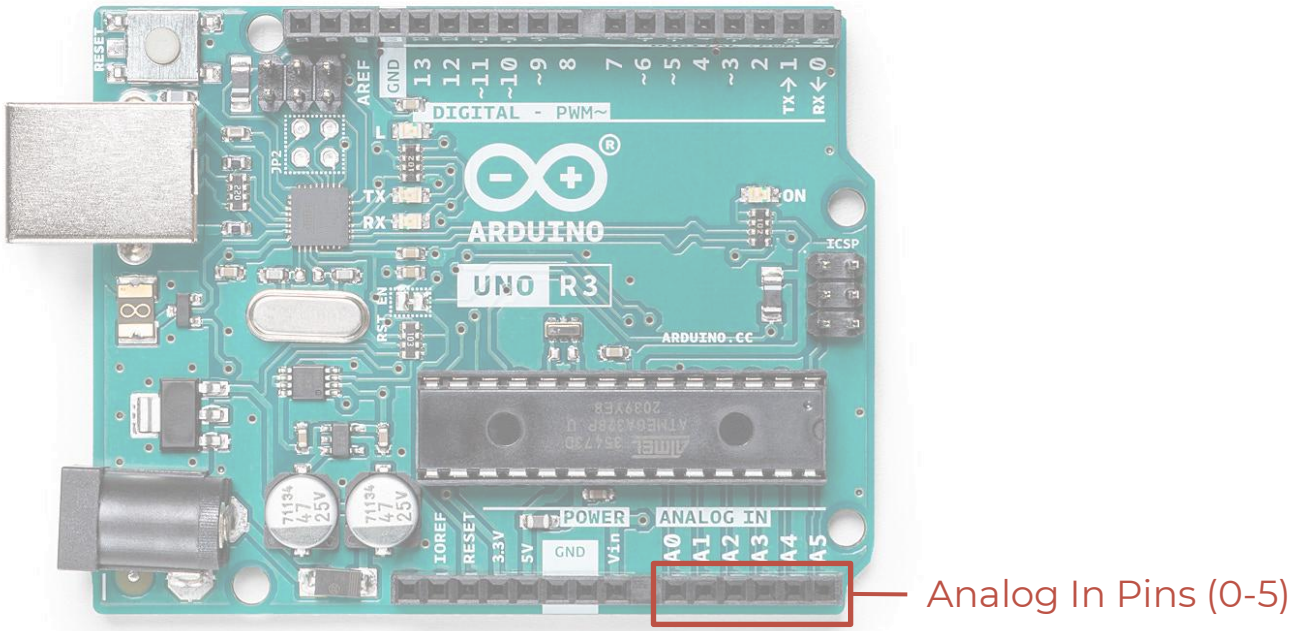
Context for Today's Lesson



Arduino Uno


Ground Pin
Analog Reference Pin
Digital I/O Pins (2-13)
Serial Out (TX)
Serial In (RX)
Reset Button
USB Port
In-Circuit Serial Programmer
ATmega16 Microcontroller
ATmega328 Microcontroller
DC Power Jack
Analog In Pins (0-5)
IO Reference Pin
Reset Pin
3.3 V Power Pin
Voltage In Pin
Ground Pins
5 V Power Pin
DIGITAL - PWM~
ARDUINO
UNO R3
POWER
ANALOG IN

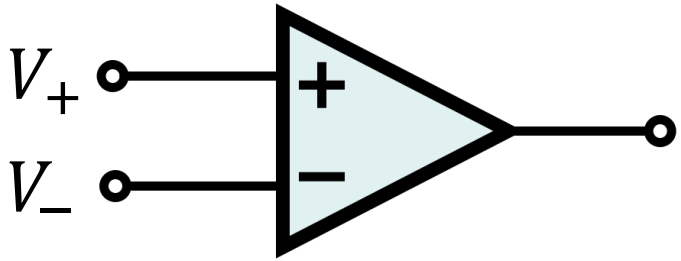
Analog to Digital Conversion



- Arduino has a 10-bit ADC for converting up to 6 analog inputs to digital
- Quantizing 0-5V (default) into 1024 discrete values
- **Do not exceed 5V with an analog input!**

Analog to Digital Conversion

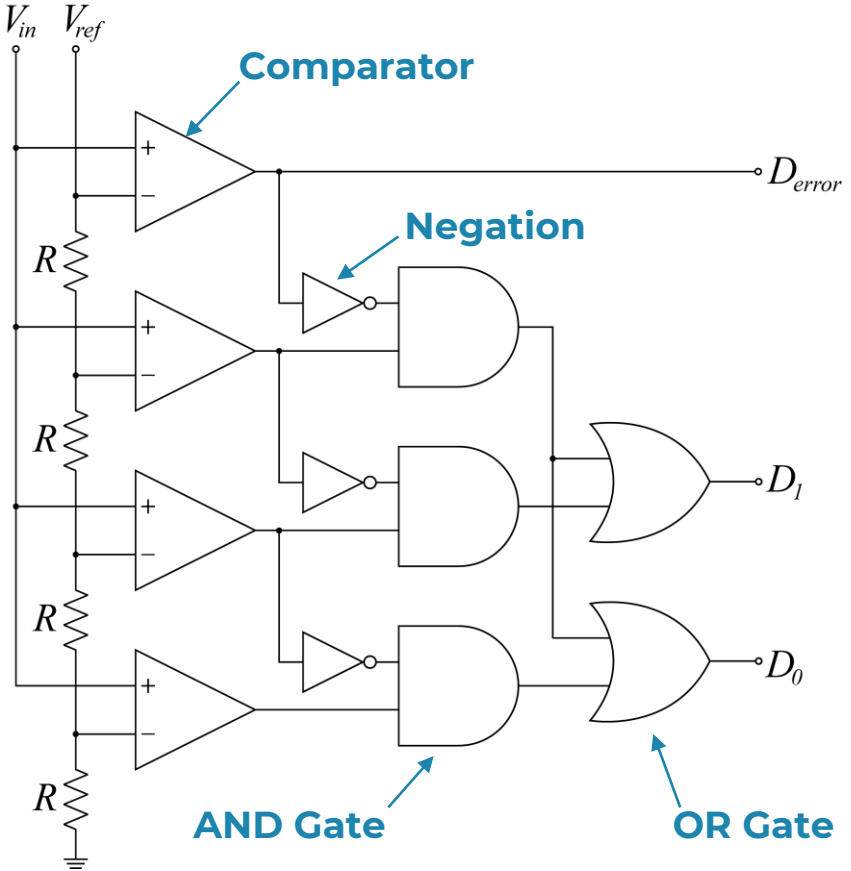
All analog to digital conversion is done by comparing the input voltage to a known **reference voltage** using a **comparator** (which could be an operational amplifier or a dedicated chip)



$$V_{out} = \begin{cases} 1, & \text{if } V_+ \geq V_- \\ 0, & \text{if } V_+ < V_- \end{cases}$$

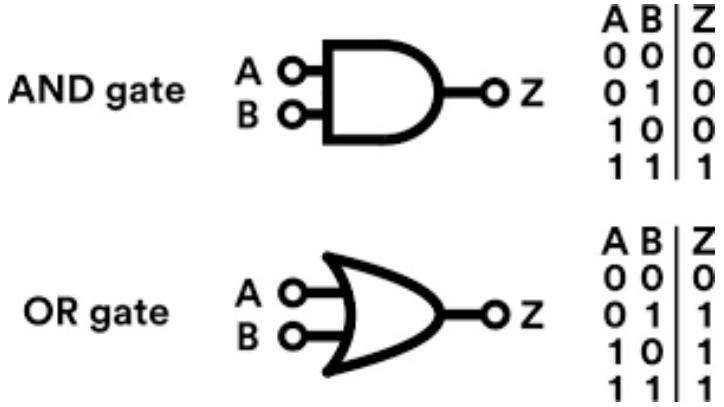
Analog to Digital Conversion

- One ADC example is the **flash converter**
- V_{in} is the **input voltage** to be measured
- V_{ref} is as **reference voltage** which is divided into equal smaller parts using a bank of resistors
- The input and divided voltages are passed through **comparators**, as well as **AND** and **OR gates** to produce a digital output

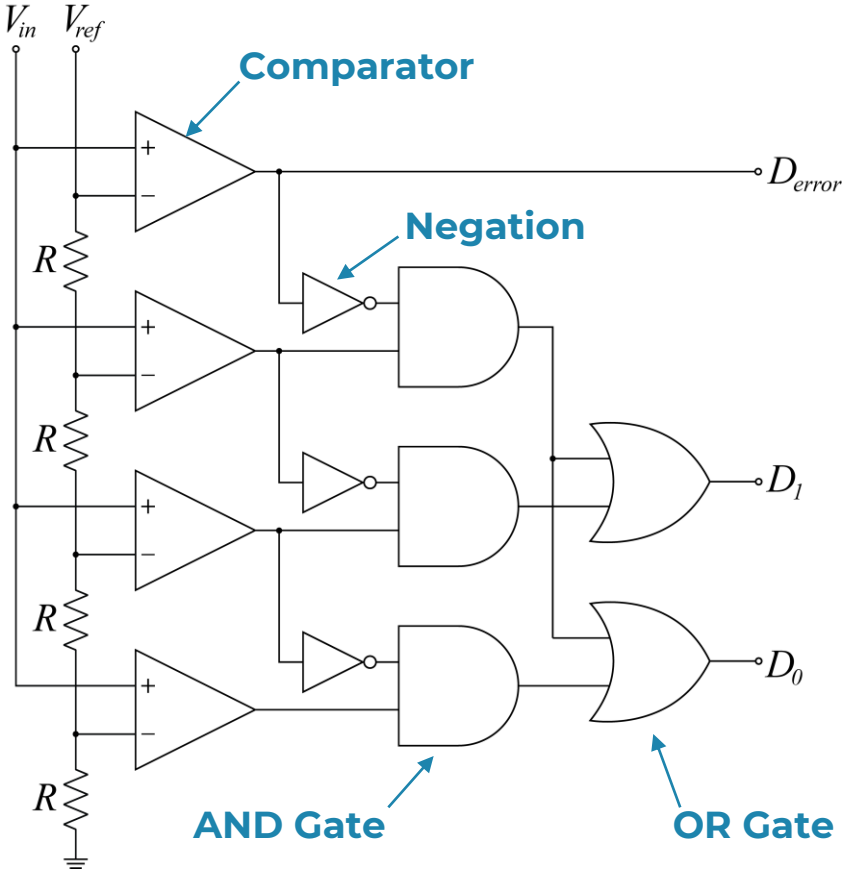


Analog to Digital Conversion

- Review of AND and OR gates:

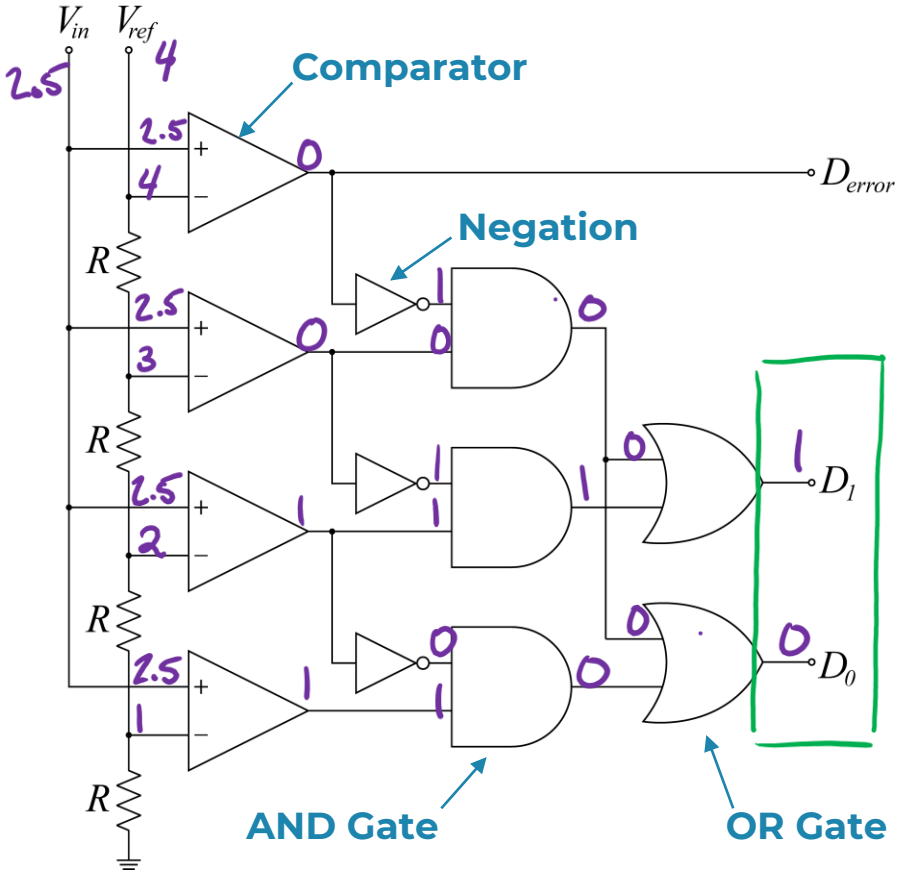


- D_0 and D_1 are the bits of the digital output
- $D_{error} = 1$ indicates that V_{in} is out of range (i.e. $V_{in} > V_{ref}$)



Analog to Digital Conversion

For the 2-bit flash converter shown, determine D_0 and D_1 if $V_{in} = 2.5$ V and $V_{ref} = 4$ V. Label the voltage across each resistor and label 1 or 0 on the output legs of each comparator, AND gate, and OR gate.



Analog to Digital Conversion

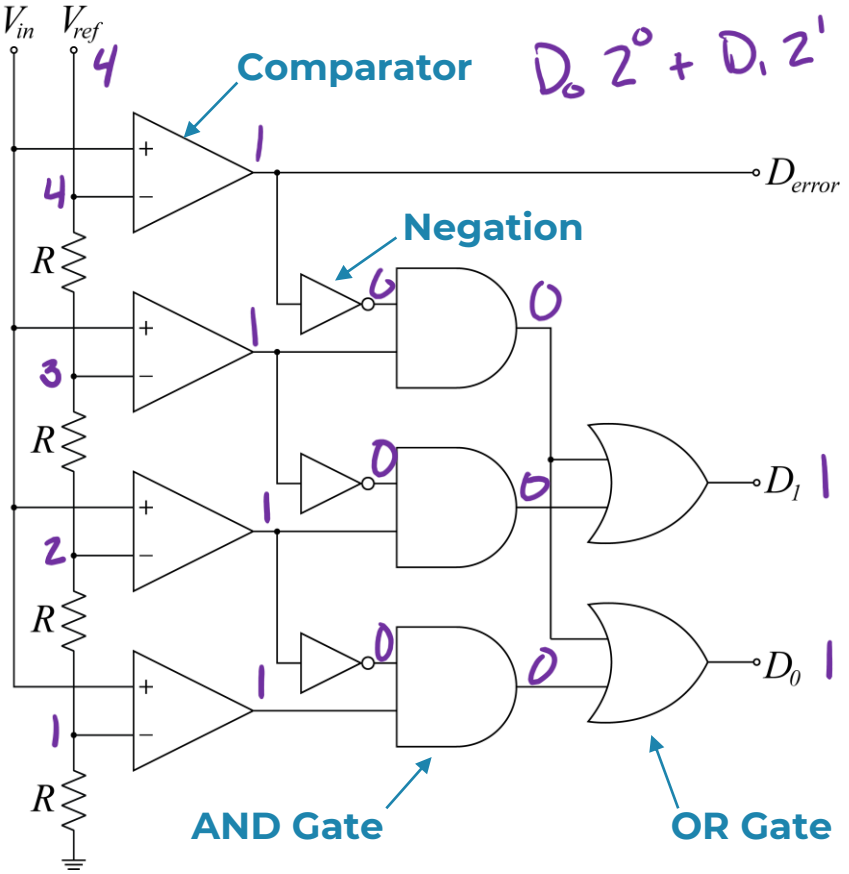
For the 2-bit flash converter shown, determine D_0 and D_1 for ranges of V_{in} given $V_{ref} = 4$ V. What numerical voltage value does the digital outputs correspond to if 0 & 4 V must be represented.

4.5

$D_0 2^0 + D_1 2^1$

$$\text{Resolution} = \frac{4}{2^2 - 1} = 1.33\text{V}$$

V_{in} [V]	D_1	D_0	D_{error}	V_{out} [V]
[0,1)	0	0	0	0
[1,2)	0	1	0	1.33
[2,3)	1	0	0	2.66
[3,4)	1	1	0	4
≥ 4		0	1	Error



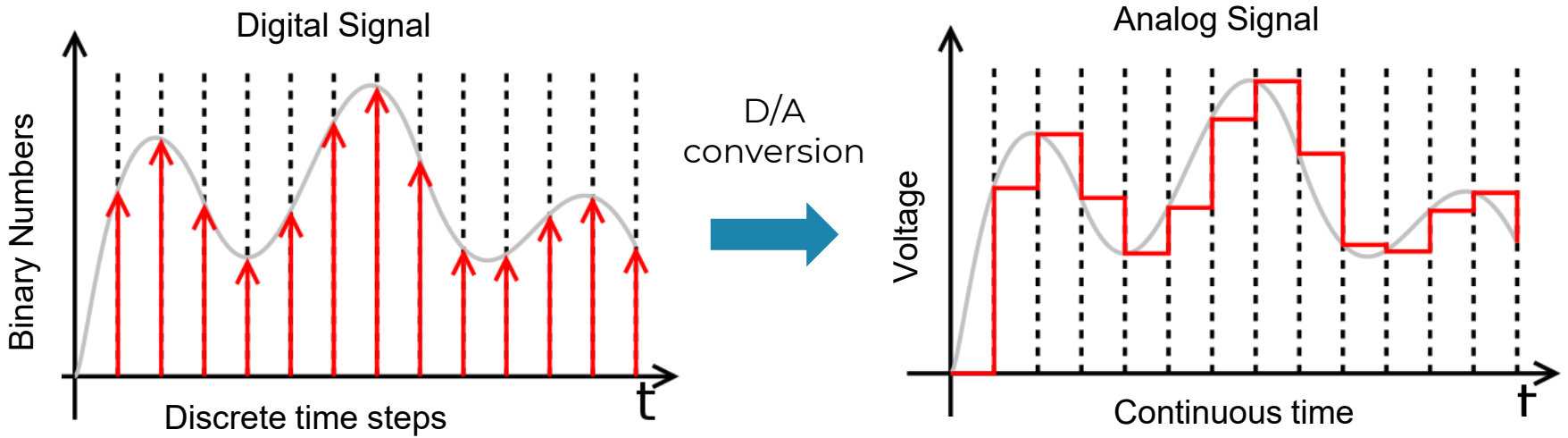
Analog to Digital Conversion

MAKE THE WORLD WORK BETTER

- Flash converters use parallel comparison and are **FAST** (hence “flash”)
- Generate the digital output almost instantaneously
- However, they are very **complex**…
  - For an n-bit converter you need 2^n resistors and 2^n comparators so they tend to be **limited in resolution**
- For this reason, there are other **less complex (but slower)** ADCs
  - Successive Approximation Register (SAR)
  - Single Slope Converter
- SAR is a popular choice for medium speed, moderate-to-high resolution applications (i.e. Arduino)

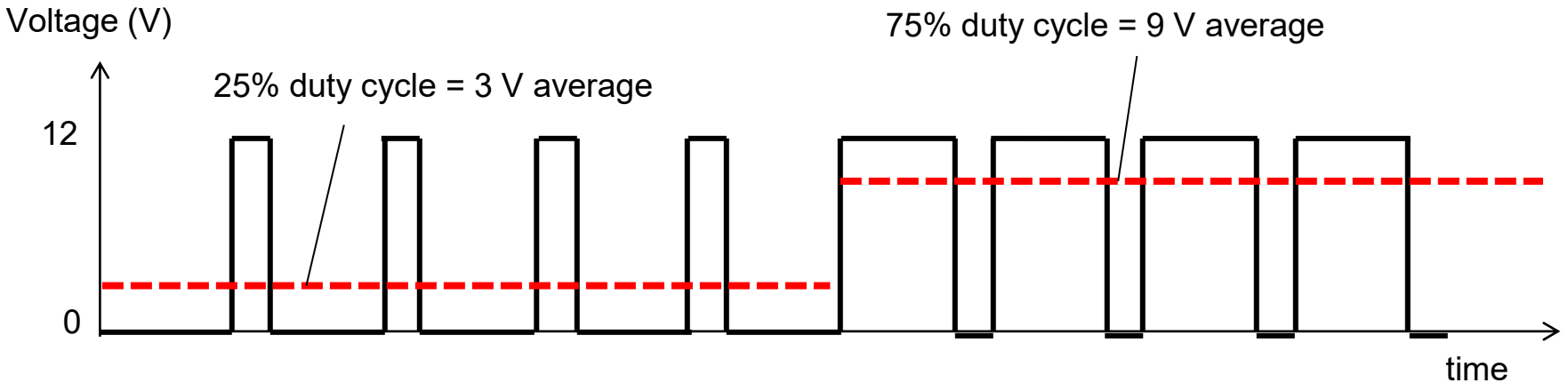
Digital to Analog Converter

- Digital to Analog Converters (DACs) take discrete digital signals and convert them to (piecewise) continuous signals (**continuous in time but still discrete in value**).
- **Analog filter** must be used to smoothen out the steps caused by quantization



D/A Conversion Using PWM

- **Pulse width modulation (PWM)** is a common way to perform D/A conversion.
- PWM adjusts the **average value** of voltage going to an electronic device by turning on and off the power to the device at a **fast rate.**
- The average value of the switching signal is **directly proportional** to the **duty cycle**
- Duty cycle = Time On/(Time On + Time Off)



D/A Conversion Using PWM

V_{supply}

V_{supply}

5V

0

0

Voltage

$V_{out} = \text{Duty Cycle} \times V_{supply}$

Time

Digital Controller

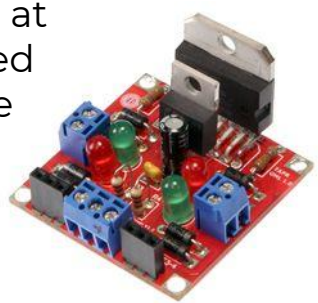
PWM Amplifier

Low-pass filter

Low power PWM signal at commanded Duty Cycle

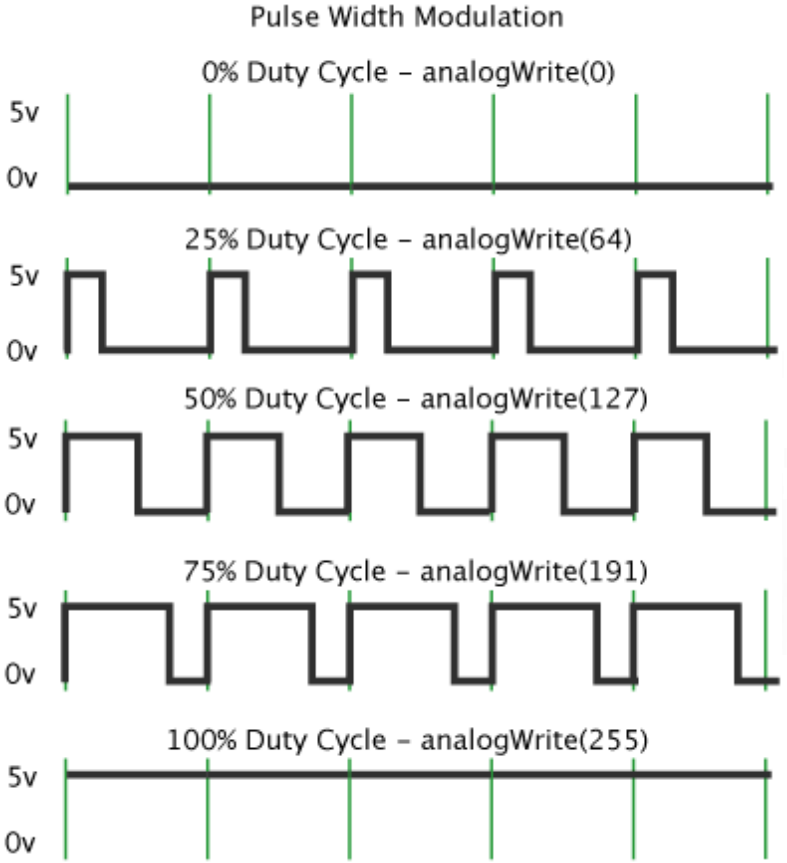
High power PWM signal at commanded Duty Cycle

Smoothened Analog Signal



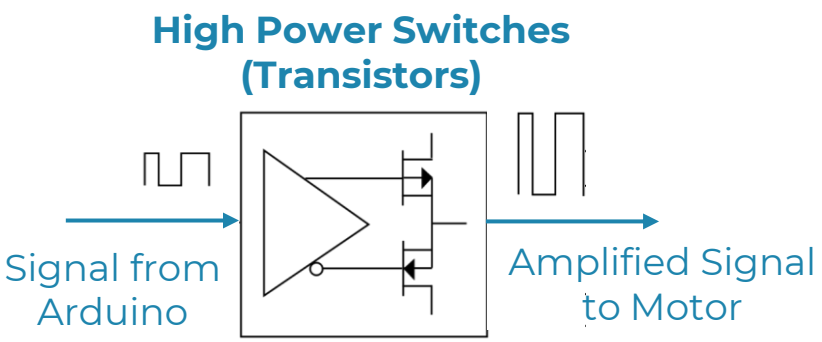
Motor acts as a low pass filter

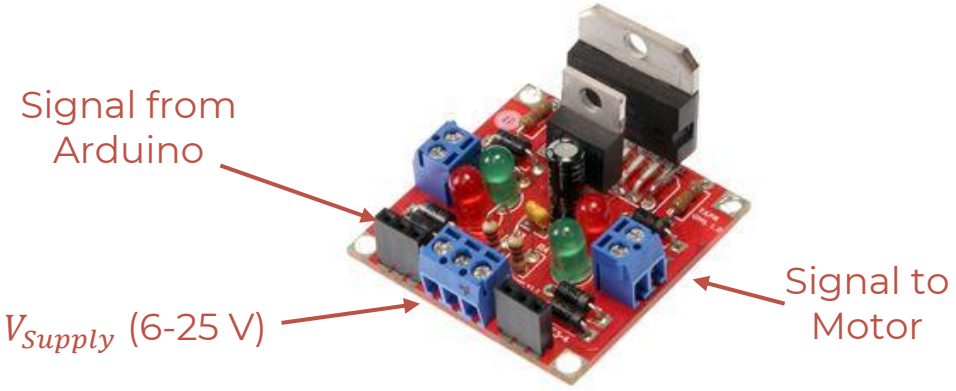
PWM Signal from Controller



- Arduino generates duty cycles proportional to discrete voltage levels (represented by binary numbers)
- Arduino has an 8-bit DAC so it can generate 256 (i.e. 0 to 255) duty cycles ranging from 0 to 100%
- The switching frequency in Arduino is 490 or 980 Hz (depends on pin) but in many other microcontrollers it is in the order of 10 kHz
- Higher switching frequency produces a **smoother** filtered signal

PWM Amplifier

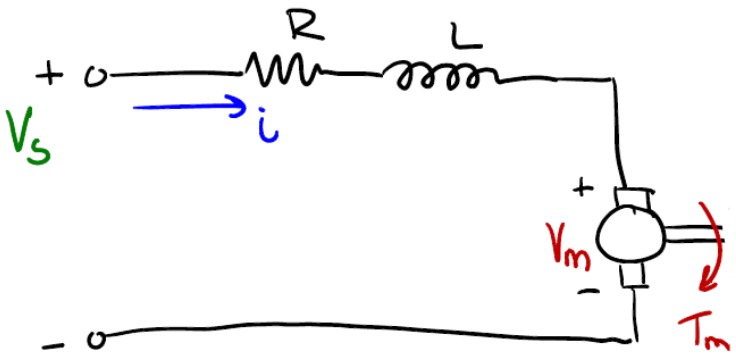




- Amplification of PWM signals is done using **transistors** to gain high power conversion efficiency
- Ideal transistors are on-off switches which have **100% efficiency**
  - Off: high voltage and zero current
  - On: high current and zero voltage
- Actual transistors are not perfect on-off switches but come close (>90% efficiency)

Motor as A Low-Pass Filter

The relationship between supply voltage (V_s) and output current (or voltage, V_R, across the resistor) of a motor is a low pass filter (even if the motion of the motor is ignored, i.e., back EMF = 0)



$$V_s = V_R + V_L = iR + L\frac{di}{dt}$$

Taking Laplace Transform (assuming zero initial conditions)

$$V_s(s) = RI(s) + sLI(s)$$
$$V_s(s) = (R + sL)I(s)$$

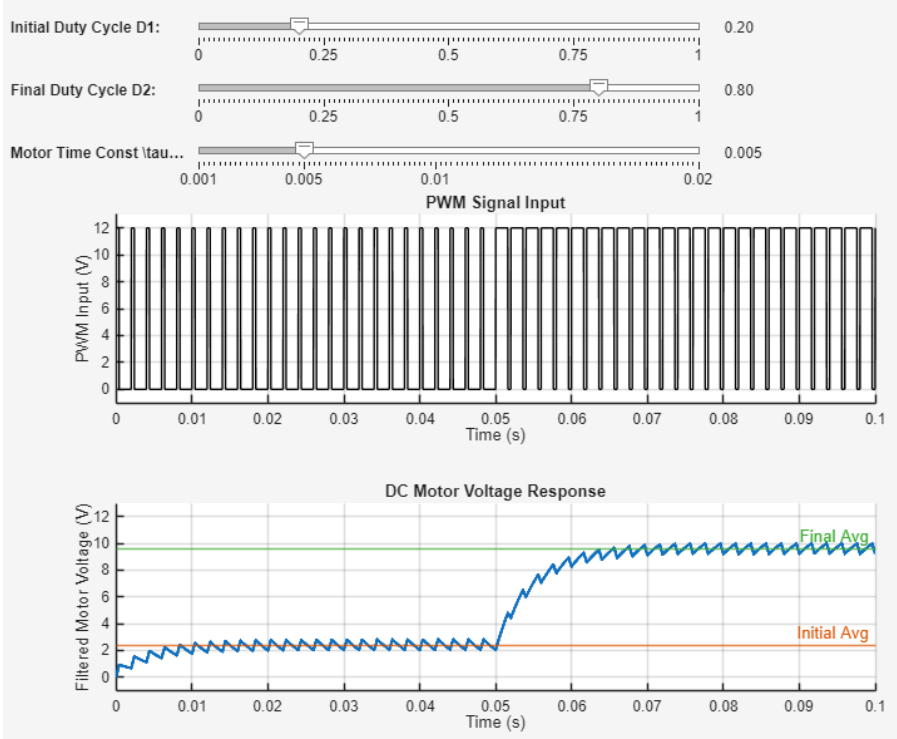
$$\Rightarrow \frac{I(s)}{V_s(s)} = \frac{1}{sL + R} = \frac{1/R}{\tau s + 1}$$

$\tau = \frac{L}{R}$ is the electrical time constant

The above implies that the electrical dynamics of a motor act as a **first-order low-pass filter**.

Motor as A Low-Pass Filter

- As a result, the motor does not respond **instantaneously** to sudden changes in supply voltage, it will be a stable exponential with a **time constant** of L/R
- Therefore, if a PWM signal is applied to a motor, the motor ends up experiencing a **filtered (or average-like)** value of current (or output voltage across its resistor)
- The average value varies in proportion with the duty cycle, as shown below



Example

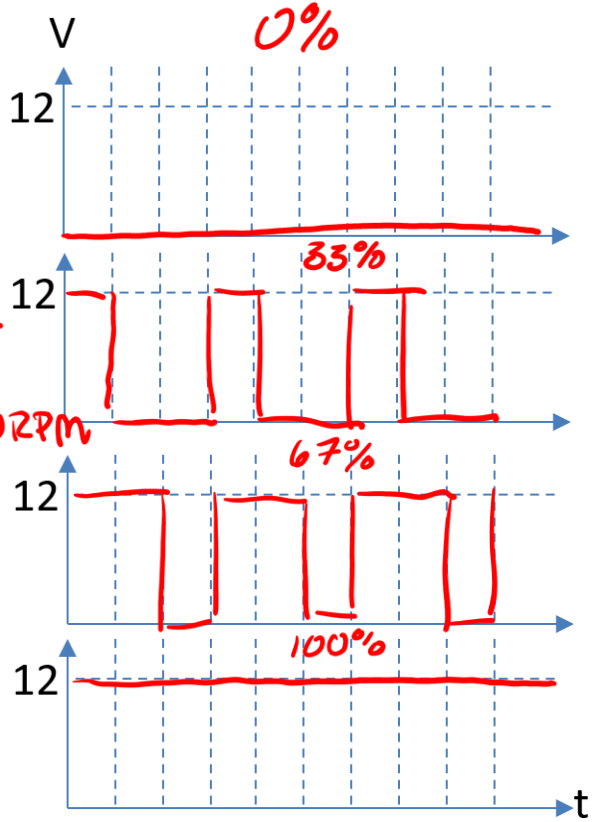
A DC motor is driven by a 2-bit PWM-based D/A converter. The max. voltage that can be supplied by the converter is 12 V, which makes the motor to turn at 3000 rpm. Determine all other values of motor speed that can be commanded by the D/A converter (based on a 0-12 V supply voltage) and sketch their corresponding PWM signals.

Duty Cycle res = $\frac{100}{2^2-1}$ = 33.33%

Voltage Res = $\frac{12}{2^2-1}$ = 4V

Speed res = $\frac{3000}{2^2-1}$ = 1000RPM

Binary Number	Duty Cycle (%)	Supply Voltage (V)	Speed (RPM)
00	0	0	0
01	33.33%	4	1000
10 11 ↓	66.67%	8	2000
	100%	12	3000



Moving Forward...

- Let's look at control!